Uppsala University

Project in Scientific Computing

Goal-Oriented Adaptive FEM for Option Pricing with Duality-Based A Posteriori Estimates

Group 19:
Anton Sjöström
Filip Marttala
Philip Isaac

1 Introduction

1.1 Background

The financial market facilitates the trading of various financial assets. These assets can come in many forms but the most common and well-known form of an asset would be a stock. While the stock market isn't entirely random, the factors influencing stock prices are numerous, complex, and often unpredictable that trying to predict them properly is practically impossible. Therefore we instead model stock prices stochastically. While there are many ways to model stock prices, and people have defined many different models, one of the most common is to model the market using geometric Brownian motions. The stock price follows a stochastic differential equation (SDE), represented as

$$dS_t = rS_t dt + \sigma S_t dW_t. \tag{1}$$

Here, W_t is a Wiener process or Brownian motion, r is the percentage drift and σ is the percentage volatility, in financial terms these would be the riskless rate of interest and the stock volatility, respectively. The SDE models the change in the price of the stock (dS_t).

An option is a financial asset that gives the holder the right, but not the obligation, to buy or sell an asset for a specified price K, usually called a strike or exercise price, at a specified point in time T, sometimes called the time of maturity. Some options allow for exercising before or at this point but in this project we focus on the European type of options which only allows for exercising at time T. Two basic options are European call and put options. In a call option, the buyer has the right (but not the obligation) to purchase the underlying asset at a predetermined strike price, K, at time T. The put option instead gives the option holder the right to sell the asset for the price K at the time T [8]. If we assume that the option holder acts rationally and only exercises the option when it is a financial gain to do so, a payout function can be defined. The payout function determines the value of the option at its time of maturity. For a European call option, for example, the payout function would be 0 if the underlying asset is worth less than the strike K since it would be a loss to buy the asset for more than it is worth. If the asset instead is worth more than the strike K then the value of the option would be the difference between the asset value and the strike K since buying it means we are trading the value of the stock for the value of K money. This payout function can then be written as $\phi(S) = \max(S - K, 0)$.

Alongside options, one important method of trading is shorting assets. This involves borrowing the asset and immediately selling it, later re-buying the asset, and returning it. This can be seen as the opposite of buying the asset because the total value of a short is $S_0 - S_T$ (S_0 is money gained from initially selling the asset and S_T is the money lost from having to re-buy and return the asset). Conversely, if one buys the asset initially and later sells it, the total value would be $S_T - S_0$. Note, that the value of a short is the negative value of buying the asset.

Options allow for more complex investing strategies, but they can also be used to reduce the risk of a current position. Even though the option can reduce risk, an option can carry risk with it as well. To reduce the risk of the change in the option's value, the stock

can be bought or sold, a strategy called *delta hedging*. This strategy involves owning an option and the underlying stock in such a way that they offset each other. For example, if we have a call option where the asset price exceeds the strike price, we can short the underlying asset to keep the payout where it is. This is because if the asset price goes down, the value of the option goes down, but the value of the short goes up. This allows us to reduce the variance of our payout, reducing risk.

Let us first create a portfolio involving a European call option and a shorting of the underlying asset S, then say that $S \gg K$, which makes the value of the call option to be approximatly $S - K$. Now, the difference in the value of the portfolio between time zero and time T would be

$$(S_T - K) - S_T - (S_0 - K) + S_0 = 0. \tag{2}$$

Therefore, we have locked the value of the portfolio by using this hedging strategy. In practice, locking a value of a portfolio is harder than this, but this is a simple example to demonstrate the concept.

The natural question that arises is what is the value of such an option? Since the payout is always non-negative, and there is always a non-zero probability that the payout is above zero, it must have a value. Further, because options allow for replicating strategies (two distinct portfolios with identical payout at some maturity time T) there is the possibility that they allow for arbitrage. We can consider the relation:

$$\max(S - K, 0) - \max(K - S, 0) - S + K = 0. \tag{3}$$

Notice that $\max(S - K, 0)$ is the payout of a European call, $\max(K - S, 0)$ is the payout of a European put, S is the payout of the asset itself and K is the payout of a zero-coupon bond. Then, if we buy a European call, short a European put, short the stock itself, and buy K bonds, then our payout is always zero. Then, the price of this strategy must also be zero in total. If it was not zero, then we could create an arbitrage by either buying this strategy (if the cost of the strategy is negative) or selling it (if the cost of the strategy is positive). Because such an arbitrage would allow for-profit at no risk it can not be allowed to exist since it would be the only strategy to bother with. This option must then have some specific price that prevents such an arbitrage from happening. Further, this price should be the average payout of the option. If it were not, then either the buyer or the seller would on average be losing money by buying or selling the options which would result in no one using the losing strategy and the existence of options becoming meaningless.

Therefore, we need some way to price these options. Because the price of an option can change rather quickly with changes in the underlying asset, it is preferable to price the option fast to keep the price accurate at the current time. Further, traders want to be able to perform trades fast and need to know the price quickly for that reason. We can do this in a few different ways. One way would be to find the average payout of the option by simulating the stock price in an attempt to estimate the stock price at the time of maturity. This is done by using the SDE defining the stock price, given in (1). Naturally, since the stock price is modeled as an SDE the resulting stock price will vary. Still, this does allow us to obtain one possible payout for the option. If we repeat

the simulation many times we can compute an approximation of the distribution of the payouts of the option, which allows us to approximate the average payout of the option. This method forms the basis for the Monte Carlo methods of pricing options. Monte Carlo methods are very popular because they are relatively easy to implement and work well for pricing options. While Monte Carlo usually has some problems that might make them unattractive, a lot of these problems are mitigated when pricing options. For example, the biggest downside of the Monte Carlo methods, their slow convergence, is mitigated by the fact that we do not need particularly high accuracy. Since currency exchange values are given with an accuracy of 1e-4 we do not need higher accuracy than that. Thus, the slow convergence of Monte Carlo is not as problematic.

If we make some assumptions about the financial market we gain another way of pricing options which is to solve the Black-Scholes equation [4]. The Black-Scholes equation is a linear partial differential equation given in its one-dimensional form in (4),

$$\begin{aligned} \frac{du}{dt} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 u}{\partial S^2} + rS\frac{\partial u}{\partial S} - ru = 0, \\ u(T) = \phi(S). \end{aligned} \tag{4}$$

Here S is the price of the underlying asset and it also acts as the spatial variable, σ is the volatility of the asset which is a measure of how much the asset price may vary over time due to random changes in the asset price. In practice, the volatility in the market may change over time however for this project we will only consider constant volatility. As long as the time of maturity is sufficiently short, this works as a good approximation, which often is made since it is easier to handle the Black-Scholes equation [8]. The risk-free interest rate is represented by r. This measures the interest rate of the market which is also the amount of monetary gain one can make without any risk involved, for example by buying zero-coupon bonds. Bonds do have risk in reality, but the risk is low enough to model it as risk-free. The function $\phi(S)$ is the payout of the option concerning the asset price S. For example, for a European call option, we would have $\phi(S) = \max(S - K, 0)$.

However, the Black-Scholes model, and by extension the Black-Scholes equation, only holds if we make some assumptions about the market as well as the assets themselves. One of the major assumptions regarding the assets is that they do not pay dividends. A dividend is a payout given to the holder of the asset. For arbitrage reasons, whenever these dividends are given out, the asset value falls which of course changes the value of the stock at the time of maturity of an option and as such also the value of the option. Major assumptions on the market assume no arbitrages, the ability to buy and sell freely as well as lack of any transaction costs [8]. While there are extensions of the model that remove or relax these assumptions (some of which will be explored in this project), these extensions typically make the equation more complex and harder to solve. Despite the above-mentioned complications, the Black-Scholes equation is still a useful model in most cases and one of the most problematic assumptions, that is no dividends, can be removed by rather minor changes to the equation, as long as we assume the dividends are continuous. In practice, however, they are not. While discrete dividends are more problematic to add to the Black-Scholes equation, it can be done, allowing us to avoid this assumption [3].

The Black-Scholes equation is important since it involves the asset price as a spatial variable. Consequently, we need to solve for option prices across various initial asset values, despite that we know the initial value of the asset and as such we only really need one point. We still need to solve for an entire domain of points however to find the option value at that one point. Another interesting property of the Black-Scholes equation is that it is a terminal-boundary value problem rather than the more common initial-boundary value problem. This is because the option value is known at the time of maturity it is the same as the payout. Otherwise, a trivial arbitrage could be made since the option could be bought at the time of maturity and immediately cash it in for more than you bought it for, or short it for the same effect if the price is greater than the payout. As is well known, in order to guarantee the uniqueness of the PDE in (4), some appropriate boundary conditions are required. While for certain options in 1D this is straightforward, it turns out to be quite a problem for many options in higher dimensions. This is another reason why Monte-Carlo methods are quite popular, since they do not aim to solve the PDE, they do not require boundary conditions which often makes the problem much easier.

Any discretization method could be used for (4), however, some may be more appropriate than others. Finite difference methods (FDM) are very popular because they have fast convergence and are relatively simple to implement. Further, many of the downsides of the finite differences are mitigated by the circumstances of the problem. One major downside of the finite differences is that it is difficult to formulate them for computational domains that are not rectangular. Since we are only interested in a small subdomain of the full computational domain and we are not limited by any physical structure of the domain we can simply choose our domain to be rectangular. This makes finite differences a highly attractive way to solve the Black-Scholes equation. While Monte-Carlo methods can often be very fast at calculating the option price itself, deterministic methods such as finite differences have the advantage that they also calculate the value for all the points around the point of interest. This means that when the solution is found it is easy to also compute the first and second derivative of the option price concerning the asset price. This is unlike Monte-Carlo which only calculates the option value in exactly one point. This means that in order to enable a Monte-Carlo method to find values for these derivatives, it must be performed many times, which comes at an increased computational cost.

Another way of solving the Black-Scholes equation is using finite element methods (FEM). FEM are very popular in many areas of computational physics due to their generality and flexibility [7]. FEM are however relatively complex to implement and as such are much less popular when easier alternatives exist, as is the case in computational finance where both Monte-Carlo and FDM perform well. Further, since options are often priced with only one underlying asset, meaning the equation has only one spatial dimension, FEM is less attractive since many of FEM's greatest strengths are only apparent in higher-dimensional problems. Despite this, there are still reasons why FEM might be worth considering for pricing options. The main reason is that if we work in more than one dimension it naturally admits unstructured meshes. This is unlike FDM, which requires structured meshes. Using unstructured meshes is advantageous when working with more complex domain shapes but in our case, it is also important because it allows some particular types of refinements [1].

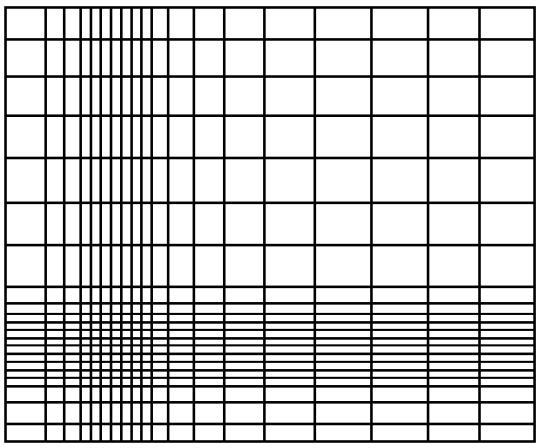

Figure 1: Refinement of a small subdomain using a structured mesh.

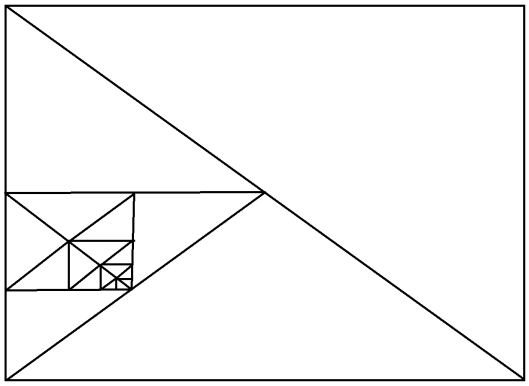

Figure 2: Refinement of a small subdomain using an unstructured mesh.

Because we are only interested in a small subdomain of the full computational domain, it is not necessary to solve the problem accurately for points far away from the point of interest. It would then be a waste of computational power to solve the equation with high accuracy in the entire domain. If we could instead focus on solving the equation accurately around the point of interest and solving the problem in the rest of the domain only accurately enough such that its error does not propagate we could save computational power. While this can be done with the structured mesh of FDM, it requires refinement along an entire axis, demonstrated in 2D in Figure 1. Even though this is significantly better than a completely uniform mesh, it is still quite wasteful spending a lot of computational resources on calculating an accurate solution for points that are not of interest. Meanwhile, the unstructured FEM mesh allows for a refinement that, for example, only focuses on the point of interest and its neighboring points, demonstrated in Figure 2. It is worth noting that these meshes are for demonstration purposes only, in practice the FEM mesh may need to refine at different parts of the domain to avoid the error to propagate.

The flexibility in mesh refinement makes FEM worth considering for solving the Black-Scholes equation, even though it's more complex than the FD method.

The natural question that arises then is how we should perform the refinement. In general, we want to refine mostly around the point of interest but we can not focus only on that point. While we do not need very high accuracy in points far away from the point of interest if the error is sufficiently large it may propagate and disturb the solution in the point of interest. Thus, we may require some refinement in the rest of the domain as well. While it is possible with prior knowledge and error estimates to approximate this beforehand, we want our method to be as general as possible. Because we want our solution quickly, often in the span of seconds, we do not have time to tailor our method or mesh to the specific problem every time. Further, since we want our solution fast, we do not want to overshoot and make our initial mesh too fine, since it will increase the computation time.

1.2 Spread option

The spread call option gives the holder the right, but not the obligation, to exchange the asset s_2 for s_1 at a price K. When K is zero, it is referred to as an exchange option. This can be also understood in terms of a call and a put option with the same maturity. A spread option would be a combined call and put where either both or none has to be exercised. The call option will be on s_1 and the put on s_2.

The spread option poses intriguing challenges due to its complex boundary conditions. Within the payout structure, a derivative discontinuity propagates along a diagonal line across the entire domain, resulting in boundaries that cannot be treated as asymptotic values. Furthermore, this discontinuity also presents challenges within the domain.

There are different ways to handle this, either by trying to find a good closed-form approximation of the price at the boundary, or by accepting that the boundary will be quite wrong and setting it far away from our points of interest. In our report, we will do both.

1.3 Basket option

A basket option involves multiple underlying assets, typically exhibiting correlations. The payout of a European basket put can be defined as $\max(K - \text{avg}(S), 0)$, where $\text{avg}(S)$ represents the average price of a collection of multiple assets.

1.4 Outline of the report

This paper has a dual focus. Firstly, we explore the efficiency of solving the Black-Scholes PDE using the Finite Element Method across various scenarios. These scenarios include the following:.

1. Spread option in 2D with constant volatility
2. Basket option in 3D with constant volatility
3. Non-linear Black-Scholes

The second main focus of this paper lies in utilizing goal-oriented adaptive mesh refinement to enhance computational speed while maintaining solution accuracy. For each scenario, we compare four types of mesh refinement. These four refinement methods are as follows.

1. Uniform mesh refinement
2. Residual-based mesh refinement
3. Local mesh refinement
4. Goal-oriented mesh refinement

The structure of the paper is as follows. First, the 2D scenario is investigated. The option is a spread option with constant volatility. Second, the 3D scenario is investigated where the option is a basket call option with equal weights. The last scenario is an option on the

non-linear Black-Scholes. In conclusion, we provide general findings and offer suggestions for future research directions.

2 The 2D problem

The formulation of the BS equation in 2D is given by the following equation

$$u_t + \frac{1}{2}\sum_{j=1}^{2}\sum_{i=1}^{2}\sigma_i\sigma_j\rho_{ij}s_is_j\frac{\partial^2 u}{\partial s_j \partial s_i} + \sum_{i=1}^{2} rs_i\frac{\partial u}{\partial s_i} - ru = 0, \tag{5}$$

where $\rho \in [-1, 1]$ and is the correlation between the two assets and $s_{1,2}$ is the price for asset one and asset two, respectively. Here, the parameters are defined as $r = 0.03$, $\sigma_1 = \sigma_2 = 0.15$, $\rho = 0.5$, $K = 0$, and $T = 1$.

The payout function for the European call spread option is given by

$$\phi(s_1, s_2) = \max(s_1 - s_2 - K, 0), \tag{6}$$

and is visualized in Figure 3.

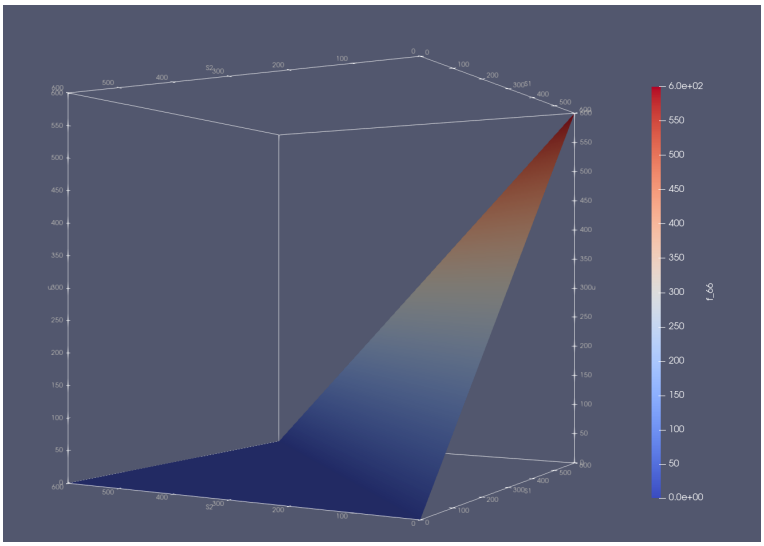

Figure 3: The payout of a spread option with K=0.

2.1 Wellposedness

Before analyzing the PDE further it is important to show that the PDE is well-posed.

Definition 1. *A PDE is well-posed if there exists a unique solution and the following relation holds:*

$$||u||^2 \leq Ke^{\alpha t}||f||^2, \tag{7}$$

where K and α are independent of data.

Conditions on the boundary and the parameters need to be fulfilled to make the PDE well-posed and these conditions are derived in this section. To acquire constant parameters in front of the derivatives in (5), the following transformation can be done

$$\begin{aligned} s_1 &= e^{x_1}, \\ s_2 &= e^{x_2}, \\ \tau &= T - t. \end{aligned} \tag{8}$$

By using the transformation (8), (5) can now be written in the following form,

$$\frac{\partial u}{\partial \tau} = (r - \frac{1}{2}\sigma_1^2)\frac{\partial u}{\partial x_1} + (r - \frac{1}{2}\sigma_2^2)\frac{\partial u}{\partial x_2} + \rho\sigma_1\sigma_2\frac{\partial^2 u}{\partial x_1 \partial x_2} + \frac{1}{2}\sigma_1\frac{\partial^2 u}{\partial x_1^2} + \frac{1}{2}\sigma_2\frac{\partial^2 u}{\partial x_2^2} - ru. \tag{9}$$

Using the substitutions

$$\begin{aligned} a &= r - \frac{1}{2}\sigma_1^2, \\ b &= r - \frac{1}{2}\sigma_2^2, \\ c &= \rho\sigma_1\sigma_2, \\ \alpha &= \frac{1}{2}\sigma_1, \\ \beta &= \frac{1}{2}\sigma_2, \end{aligned} \tag{10}$$

we can rewrite (9) as

$$\frac{\partial u}{\partial \tau} = a\frac{\partial u}{\partial x_1} + b\frac{\partial u}{\partial x_2} + c\frac{\partial^2 u}{\partial x_1 \partial x_2} + \alpha\frac{\partial^2 u}{\partial x_1^2} + \beta\frac{\partial^2 u}{\partial x_2^2} - ru. \tag{11}$$

To show the well-posedness of this equation, the energy method is applied. By multiplying with u and integrating by parts we obtain

$$\begin{aligned} -(u, u_t) + \int_{\Gamma_2} uau|_{x_w}^{x_e} dx_2 - (u_{x_1}, au) + \int_{\Gamma_1} ubu|_{x_s}^{x_n} dx_1 - (u_{x_2}, bu) \\ + \int_{\Gamma_2} ucu_{x_2}|_{x_w}^{x_e} dx_2 - (u_{x_1}, cu_{x_2}) + \int_{\Gamma_2} u\alpha u_{x_1}|_{x_w}^{x_e} dx_2 - (u_{x_1}, \alpha u_{x_1}) \\ + \int_{\Gamma_1} u\beta u_{x_2}|_{x_s}^{x_n} dx_1 - (u_{x_2}, \beta u_{x_2}) - r||u|| = 0, \end{aligned} \tag{12}$$

where u_{x_1} denotes the derivative with respect to x_1, Γ_1 denotes the north and south boundaries and Γ_2 denotes the east and west boundaries of the domain, visualized in Figure 4. We can rewrite (12) by using the matrix A and the vector w, defined in (13) and (14) respectively.

$$A = \begin{pmatrix} \alpha & \frac{c}{2} \\ \frac{c}{2} & \beta \end{pmatrix}, \tag{13}$$

$$w = [u_{x_1}, u_{x_2}]^\intercal. \tag{14}$$

This gives

$$-(u, u_t) - (u_{x_1}, au) - (u_{x_2}, bu) - (w, Aw) - r||u|| + \mathrm{BT} = 0, \tag{15}$$

where BT is given by

$$\mathrm{BT} = \int_{\Gamma_2} uau|_{x_w}^{x_e} dx_2 + \int_{\Gamma_1} ubu|_{x_s}^{x_n} dx_1 + \int_{\Gamma_2} ucu_{x_2}|_{x_w}^{x_e} dx_2 + \int_{\Gamma_2} u\alpha u_{x_1}|_{x_w}^{x_e} dx_2 + \int_{\Gamma_1} u\beta u_{x_2}|_{x_s}^{x_n} dx_1. \tag{16}$$

By adding to (15) its conjugate transpose, given in (17),

$$-(u, u_t) + (u_{x_1}, a^* u) + (u_{x_2}, b^* u) - (w, A^* w) - r^* ||u|| + \mathrm{BT}^* = 0, \tag{17}$$

We obtain (18). Note that BT is a real scalar and coincides with its conjugate transpose.

$$\frac{\partial ||u||^2}{\partial \tau} = -(u_{x_1}, (a - a^*)u) - (u_{x_2}, (b - b^*)u) - (w, (A + A^*)w) - (r + r^*)||u|| + 2BT. \tag{18}$$

By looking at (18), three terms can already be neglected since a and b are real coefficients, which makes the terms disappear, and r is real and non-negative, which makes the term $-r||u||$ strictly negative. The remaining part is then to show that the boundary terms and the w-term are negative which would lead to that the energy of the system is never going to increase. To make the w-term fulfill the criteria, A needs to be positive semi-definite. The eigenvalues can be evaluated, which gives

$$\lambda = \frac{1}{2}(\alpha + \beta) \pm \sqrt{\frac{1}{4}(\alpha + \beta)^2 - (\alpha\beta - \frac{1}{2}c^2)}. \tag{19}$$

For A to be positive semi-definite, the quantity inside the root has to be positive and less than the quantity outside the root such that the eigenvalues are real and non-negative. Start by looking at the quantity inside the root. It is necessary that $(\alpha - \beta)^2 \geq -c^2$ which always is true. Then by analyzing the expression outside the root, it would be necessary that

$$\frac{(\alpha + \beta)^2}{4} \geq \frac{(\alpha + \beta)^2}{4} - \left(\alpha\beta - \frac{c^2}{4}\right). \tag{20}$$

This gives

$$0 \geq -\left(\alpha\beta - \frac{c^2}{4}\right). \tag{21}$$

Finally, we obtain the condition

$$\alpha\beta \geq \frac{c^2}{4}, \tag{22}$$

which, using the definitions in (10) can be rewritten as

$$\frac{\sigma_1^2\sigma_2^2}{4} \geq \frac{\rho^2\sigma_1^2\sigma_2^2}{4}. \tag{23}$$

Condition (23) holds true as long as $-1 \leq \rho \leq 1$, which is always true by definition.

The boundary conditions now have to be imposed such that the boundary terms vanish in (18). By either setting Dirichlet or Neumann conditions on the boundaries, the PDE becomes well-posed. It is not necessary to set boundary conditions on the west and south since the variables a or b are zero there before the transformation, but boundary conditions are necessary on the east and north boundaries.

2.2 Primal Problem

Definition 2. *Define the inner product* $(u,v)_p = \int_\Omega uvdx = (u,v)_{L^2(\Omega)}$.

To solve the primal problem based on (5), the weak formulation has to be derived, which reads as follows. Let $P_n(\Omega)$ be the space of n:th degree polynomials on Ω, find $u \in V$, where $V := \{v : v \in C^0(\Omega), v|_k \in P_2(k)\ \forall k \in \tau_h\}$, such that, for all $v \in V$,

$$\begin{aligned}\left(\frac{\partial u}{dt}, v\right)_p - \frac{1}{2}\left(\frac{\partial u}{ds_1}, \frac{\partial}{\partial s_1}(\sigma_1^2 s_1^2 v)\right)_p + \int_{\Gamma_1} \frac{1}{2}\sigma_1^2 s_1^2 \frac{\partial u}{\partial s_1} v ds \\ -\rho\left(\frac{\partial u}{\partial s_2}, \frac{\partial}{\partial s_1}(\sigma_1\sigma_2 s_1 s_2 v)\right)_p + \int_{\Gamma_1} \rho\sigma_1\sigma_2 s_1 s_2 \frac{\partial u}{\partial s_2} v ds \\ -\frac{1}{2}\left(\frac{\partial u}{\partial s_2}, \frac{\partial}{\partial s_2}(\sigma_2^2 s_2^2 v)\right)_p + \int_{\Gamma_2} \frac{1}{2}\sigma_2^2 s_2^2 \frac{\partial u}{\partial s_2} v ds \\ +rs_1\left(\frac{\partial u}{\partial s_1}, v\right)_p + rs_2\left(\frac{\partial u}{\partial s_2}, v\right)_p - r(u,v)_p = 0.\end{aligned} \tag{24}$$

Utilizing this weak formulation in conjunction with the Crank-Nicholson time discretization scheme and the substitutions, $\tau = T - t$ and $\frac{\partial}{\partial t} \to \frac{\partial}{\partial \tau} = -\frac{\partial}{\partial t}$ the computational scheme can be implemented in FEniCS, a computational finite element method library that will be further discussed in Chapter 2.8. We apply Dirichlet boundary conditions,

Figure 4: Clarifying the Dirichlet Boundary Conditions.

with a homogeneous condition set on the western boundary, see Figure 4. The southern boundary condition aligns with a 1D European call option, allowing the placement of the closed-form solution for the European call option. Setting boundary conditions on the eastern and northern boundaries is more challenging. An alternative is to apply a closed-form approximation as a boundary condition, in the hope of diffusing the error. In this case, we opt for Kirk's formula as the chosen approximation (see 2.2.1).

2.2.1 Kirk's formula

To set the east and north boundary conditions, Kirk's formula [6] is implemented on that boundary and has the following form,

$$
\begin{aligned}
\sigma &= \sqrt{\sigma_1^2 + \sigma_2^2\left(\frac{s_2}{s_2+K}\right)^2 - 2\rho\sigma_1\sigma_2\frac{s_2}{s_2+K}}, \\
d_1 &= \frac{\log\left(\frac{s_1}{s_2+K}\right) + \left(\frac{\sigma^2}{2}\right)(T-t)}{\sigma\sqrt{T-t}}, \\
d_2 &= \frac{\log\left(\frac{s_1}{s_2+K}\right) - \left(\frac{\sigma^2}{2}\right)(T-t)}{\sigma\sqrt{T-t}} \\
&= d_1 - \sigma\sqrt{T-t}, \\
u &\approx s_1 N(d_1) - (s_2 + Ke^{-r(T-t)})N(d_2).
\end{aligned}
\tag{25}
$$

Where $N(x)$ is the cumulative standard normal density function. Note that this is a closed-form approximation of the value of a spread option, however, we will only use this for the east and north boundaries.

2.3 Dual Problem

Definition 3. *For a linear operator $\mathcal{L} : V \to V$, its dual operator $\mathcal{L}^*$ is defined such that $(\mathcal{L}u, v) = (u, \mathcal{L}^* v) \quad \forall u, v \in V$.*

Definition 4. *Define the inner product $(u, v)_d = \int_0^T \int_\Omega uv dx dt = (u, v)_{L^2([0,t]\times\Omega)}$.*

To find our dual problem we need to find the dual operator to the primal operator $\mathcal{L}$. According to (5) we have our primal operator $\mathcal{L}u$ given by

$$
\mathcal{L}u = \frac{\partial u}{\partial t} + \frac{1}{2}\sigma_1^2 s_1^2 \frac{\partial^2 u}{\partial s_1^2} + \rho\sigma_1\sigma_2 s_1 s_2 \frac{\partial^2 u}{\partial s_1 \partial s_2} + \frac{1}{2}\sigma_2^2 s_2^2 \frac{\partial^2 u}{\partial s_2^2} + r s_1 \frac{\partial u}{\partial s_1} + r s_2 \frac{\partial u}{\partial s_2} - ru, \tag{26}
$$

where all the boundary conditions are Dirichlet boundary conditions. In order to find our corresponding dual operator we set up the expression

$$
(\mathcal{L}e, z), \tag{27}
$$

where $e = u - u_h$ and z will be the unknown in our dual problem. Since the boundary conditions are Dirichlet, the boundary conditions for the error e are all homogeneous. The problem can then be described as

$$
\begin{aligned}
(\mathcal{L}e, z) = \left(\frac{\partial e}{\partial t}, z\right)_d + \frac{1}{2}\left(\sigma_1^2 s_1^2 \frac{\partial^2 e}{\partial s_1^2}, z\right)_d + \rho\left(\sigma_1\sigma_2 s_1 s_2 \frac{\partial^2 e}{\partial s_1 \partial s_2}, z\right)_d + \frac{1}{2}\left(\sigma_2^2 s_2^2 \frac{\partial^2 e}{\partial s_2^2}, z\right)_d \\
+ r\left(s_1 \frac{\partial e}{\partial s_1}, z\right)_d + r\left(s_2 \frac{\partial e}{\partial s_2}, z\right)_d - r\,(e, z)_d\,.
\end{aligned}
\tag{28}
$$

By using integration by parts (28) can then be expressed in the following way,

$$(\mathcal{L}e, z) = -\left(e, \frac{\partial z}{\partial t}\right)_d + \frac{1}{2}\left(e, \frac{\partial^2(\sigma_1^2 s_1^2 z)}{\partial s_1^2}\right)_d + \rho\left(e, \frac{\partial^2(\sigma_1\sigma_2 s_1 s_2 z)}{\partial s_1 \partial s_2}\right)_d + \frac{1}{2}\left(e, \frac{\partial^2(\sigma_2^2 s_2^2 z)}{\partial^2 s_2}\right)_d$$
$$-r\left(e, \frac{\partial(s_1 z)}{\partial s_1}\right)_d - r\left(e, \frac{\partial(s_2 z)}{\partial s_2}\right)_d - r\,(e, z)_d + \mathrm{BT}. \tag{29}$$

We see that if we can eliminate all boundary terms by setting appropriate boundary conditions for z then we obtain the dual operator in the form

$$\mathcal{L}^* z = -\frac{\partial z}{\partial t} + \frac{1}{2}\frac{\partial^2(\sigma_1^2 s_1^2 z)}{\partial s_1^2} + \rho\frac{\partial^2(\sigma_1\sigma_2 s_1 s_2 z)}{\partial s_1 \partial s_2} + \frac{1}{2}\frac{\partial^2(\sigma_2^2 s_2^2 z)}{\partial s_2^2}$$
$$-r\frac{\partial(s_1 z)}{\partial s_1} - r\frac{\partial(s_2 z)}{\partial s_2} - rz. \tag{30}$$

The boundary terms in (29) are given by

$$\mathrm{BT} = \int_\Omega ezdx\Big|_0^T + \int_0^T\int_{\Gamma_1} \frac{1}{2}\sigma_1^2 s_1^2 \frac{\partial e}{\partial s_1} zdsdt$$
$$+\int_0^T\int_{\Gamma_1} \rho\sigma_1\sigma_2 s_1 s_2 \frac{\partial e}{\partial s_2} zdsdt + \int_0^T\int_{\Gamma_2} \frac{1}{2}\sigma_2^2 s_2^2 \frac{\partial e}{\partial s_2} zdsdt. \tag{31}$$

Since a terminal condition is given, $e(T) = 0$, It is clear that setting homogeneous Dirichlet conditions on every boundary for z as well as an initial condition $z(0) = 0$ eliminates all boundary terms.

The dual problem is then given by

$$\begin{aligned} -\frac{\partial z}{\partial t} + \frac{1}{2}\frac{\partial^2(\sigma_1^2 s_1^2 z)}{\partial s_1^2} + \rho\frac{\partial^2(\sigma_1\sigma_2 s_1 s_2 z)}{\partial s_1 \partial s_2} & \\ +\frac{1}{2}\frac{\partial^2(\sigma_2^2 s_2^2 z)}{\partial s_2^2} - r\frac{\partial(s_1 z)}{\partial s_1} - r\frac{\partial(s_2 z)}{\partial s_2} - rz = \Psi, & \quad (t, s_1, s_2) \in [0, T] \times \Omega, \\ z = 0, & \quad \text{on } \partial\Omega, \\ z = 0, & \quad t = 0, \end{aligned} \tag{32}$$

where Ψ is some arbitrary forcing function. Now, using the Crank-Nicolson scheme the fully discretized problem can be implemented in FEniCS.

2.4 A posteriori error estimate

To get an a posteriori error estimate, the following error operator J is defined.

Definition 5. *An a posteriori estimate can be defined as*

$$J(e) = (e, \Psi)_d = \int_0^T\int_\Omega e\Psi dx. \tag{33}$$

Where Ψ is then a weighting function applied to the error. We note that we can define the dual problem such that it is given by $\mathcal{L}^* u = \Psi$. Using the linear operators and the definition of an adjoint we can get the following expression,

$$J(e) = (e, \Psi)_d = (e, \mathcal{L}^* z)_d = (\mathcal{L} e, z)_d = (\mathcal{L}(u - u_h), z)_d = (-R, z)_d \,, \tag{34}$$

where R is the residual. Since the residual is orthogonal to any element in the finite element space their inner product is zero, and can thus be added to (34). Note that the interpolant of the solution z, denoted πz, is in the finite element space.

$$J(e) = (-R, (z - \pi z))_d \,. \tag{35}$$

The integral can be split into a sum of integrals on the triangulation and by using the Cauchy-Schwartz inequality and the interpolation estimate the following error estimate can be obtained,

$$|J(e)| \leq C \int_0^T \sum_{k \in \tau_k} h_k ||R|| ||\nabla z|| dt, \tag{36}$$

where C is an interpolation constant, and h_k is a measure of the size of the element.

2.5 Target function

The target function Ψ plays an important part in how the dual solution will behave. If we use the error as a target, then J(e) will be the L_2-norm of the error in the domain. When the target function is a constant value of 1, then J(e) will be the average error in the domain. In the case of a discontinuous target function, taking the value of 1 within a subset of the domain and 0 elsewhere, J(e) will be the average error in the subset. Finally, when the target function is a Gaussian distribution centered around a point of interest and with a defined width, it corresponds to a specific weighted error. The latter is the function that we have used in the project as the target function. Furthermore, this identical function has been selected for the purpose of local refinement.

2.6 Δ and Γ

It is not only the price of the option that is of importance. It is also instructive to understand how the price of the option changes when the underlying assets change. This is where Δ and Γ come into play. The first derivative of the option price with respect to one of the assets is called Δ. Similarly, Γ is the second derivative with respect to the assets.

To be able to compute these from our piecewise polynomial solution we need to project it onto a weak version of the derivatives. It is possible to calculate Δ by solving the first system of equations in (37) and Γ can be calculated by solving the second system. Here, Δ or Γ are represented by the vector w and u is the solution. The basis functions are given by φ and φ' are the first derivatives of the basis functions. The matrix C is used

when calculating Δ and A is used when calculating Γ.

$$
\begin{aligned}
C &= \sum_{i=1}^{n} \varphi_i' \varphi_j, \\
A &= \sum_{i=1}^{n} \varphi_i' \varphi_j', \\
M &= \sum_{i=1}^{n} \varphi_i \varphi_j, \\
Cw_\Delta &= Mu, \\
Aw_\Gamma &= Mu.
\end{aligned}
\tag{37}
$$

Here w_Δ is the L_2-projection of the first derivative onto the finite element space and w_Γ is the projection of the second derivative.

2.7 Mesh refinement

The mesh is used to define our system of equations. More nodes in the mesh, or the finer it is, the larger the system of equations becomes. If the system becomes too large, computing the solution becomes prohibitively slow. Conversely, if the mesh is too coarse, the solution might not attain the required level of accuracy.

There is a trade-off between accuracy and speed and mesh refinement tries handle this by beginning with a coarse mesh and then iteratively refining the mesh to increase the accuracy. This might sound ineffective since the equation needs to be solved multiple times, however, since the time it takes to solve the system increases quickly when the mesh is refined, particularly in higher dimensional problems, it is important that we use the coarsest mesh possible that computes a solution with sufficient accuracy.

How do we know when to stop the refinement process? One simple way to do it is to compare the difference between the old solution and the new, and if the difference is below a certain threshold we terminate. This does not guarantee that the accuracy of the solution is below the threshold but it can be used as an indicator.

In order to refine the mesh we must first decide which elements should be refined. The elements that are the top contributors to the error, according to an error function which is different for the different methods. For uniform refinement all elements are refined, for the residual-based refinement, the top 10% contributors to the residual are refined. The threshold doesn't need to be 10%. Decreasing it gives a more focused refinement, reducing the risk of refining unnecessary elements. This is at the cost of possibly requiring more iterations since we refine fewer elements per iteration. Increasing the threshold gives the opposite effect, a less focused refinement which may refine elements unnecessarily but may require fewer iterations if a large part of the domain needs to be refined. Since we do not know ahead of time what the optimal refinement percentage is for a specific problem, we simply choose 10% as a middle ground. For local refinement, the top 10% closest to the interesting point are refined. For the goal-oriented, the top 10% contributors to the error

estimate in (36) are refined. Later in the report, we compare which of these refinements performs the best.

The goal-oriented mesh refinement can be written in pseudocode in this manner:

Algorithm 1 Goal-oriented refinement

1. Initialize a mesh and set TOL > 0.
2. Solve the primal problem
3. Solve the dual problem
4. For every element K in the mesh, compute the a posteriori error estimator η_k
5. If error $\leq$ TOL: Stop.
6. If error $>$ TOL: Refine 10% of elements with largest η_k
7. Go to step 2.

2.8 FEniCS

FEniCS is an open-source Finite Element Method library [2] which can be used with Python as an interface. In this section, we describe how to use FEniCS to implement the Finite Element Methods discussed in the report.

To start, we need to formulate the correct weak form by hand and choose which time-stepping scheme we would like to use. Then, write the weak form with that time-stepping scheme into FEniCS. FEniCS can then create all necessary matrices and have built-in solvers to solve the system of equations. There are two different classes of solvers that could be chosen between, direct solvers and iterative solvers. In these classes, there exist various implementations.

FEniCS can also create simple meshes and can handle the mesh refinements themselves. All it needs is which elements to refine. The choice of basis functions is also easy in FEniCS. In the project, piece-wise quadratic basis functions are used.

3 The 3D problem

3.1 Primal problem

The primal problem for the 3D case is defined in the following way

$$u_t + \frac{1}{2}\sum_{j=1}^{3}\sum_{i=1}^{3}\sigma_i\sigma_j\rho_{ij}s_is_j\frac{\partial^2 u}{\partial s_j\partial s_i} + \sum_{i=1}^{3} rs_i\frac{\partial u}{\partial s_i} - ru = 0. \tag{38}$$

The parameters are $r = 0.03$, $\sigma_1 = \sigma_2 = \sigma_3 = 0.15$, $\rho_{1,2,3} = 0.5$, $K = 0$, and $T = 1$. The weak formulation is similar to the 2D case, and the scheme can be written with the Crank-Nicolson method as the time-stepping scheme before it is implemented in FEniCS.

3.2 Dual problem

The dual problem is defined as

$$-z_t + \frac{1}{2}\sum_{j=1}^{3}\sum_{i=1}^{3}\frac{\partial^2}{\partial s_j \partial s_i}(\sigma_i\sigma_j\rho_{ij}s_i s_j z) - \sum_{i=1}^{3}\frac{\partial}{\partial s_i}(r s_i z) - rz = \Psi, \tag{39}$$

which can be implemented as well with the Crank-Nicolson scheme in FEniCS.

4 Non-linear problem

There exist multiple non-linear extensions to the standard Black-Scholes equation. One of the extensions is to include transaction costs and risk adversion in the model. This extension adds a non-linear term to the diffusion term [5]. The non-linear equation becomes as follows

$$\begin{aligned} u_t + \frac{1}{2}\left[1 + \Theta\left(a^2 e^{r(T-t)}\sum_{j=1}^{2}\sum_{i=1}^{2}\sigma_i\sigma_j\rho_{ij}s_i s_j\frac{\partial^2 u}{\partial s_j \partial s_i}\right)\sum_{j=1}^{2}\sum_{i=1}^{2}\sigma_i\sigma_j\rho_{ij}s_i s_j\frac{\partial^2 u}{\partial s_j \partial s_i}\right] \\ + \sum_{i=1}^{2} r s_i \frac{\partial u}{\partial s_i} - r = 0. \end{aligned} \tag{40}$$

However, $\Theta(x) \approx x$ for asymptotic values are therefore just replaced by unity. Here the variable a is a non-negative constant that incorporates both transaction cost and risk aversion.

4.1 Frozen coefficients

To be able to solve (40) using FEM, the PDE has to be linearized. One way to do it is to treat the second derivative in the Θ function explicitly. To treat this explicitly, the solution of the last time step is used to express one of the second derivatives. Then the solution is achieved by solving for the next time step and by replacing the coefficient with the new one. If the time step is small enough, this method can be used.

4.2 Boundary conditions and payout

As a test problem for the non-linear case, we choose a spread option in 2D for the non-linear case. To make the boundary conditions simple we have chosen to use the discounted payout as the boundary conditions. The payout is defined in (41).

$$\phi(s_1, s_2) = \max\left(\frac{1}{2}s_1 + \frac{1}{2}s_2 - K, 0\right). \tag{41}$$

5 Results and Discussion

The numerical experiments are done with the same computer (CPU: AMD Ryzen 7 5800X 8-Core Processor). We also make a comparison to some of the methods in the Benchop paper [9] and then the methods are also run on the same computer but using the MATLAB software.

5.1 2D Results

5.1.1 Accuracy, time and break conditions

For the 2D case, the results are presented for the point $(s_1, s_2) = (100, 100)$, which is on the strike, since K is 0. The shape of both the primal and the dual solutions are displayed in Figures 5 and 6, correspondingly.

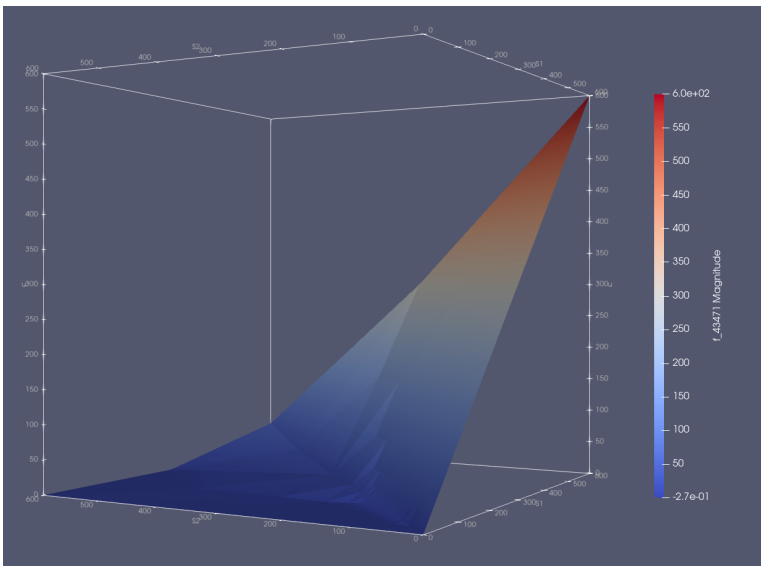

Figure 5: The solution of the primal problem.

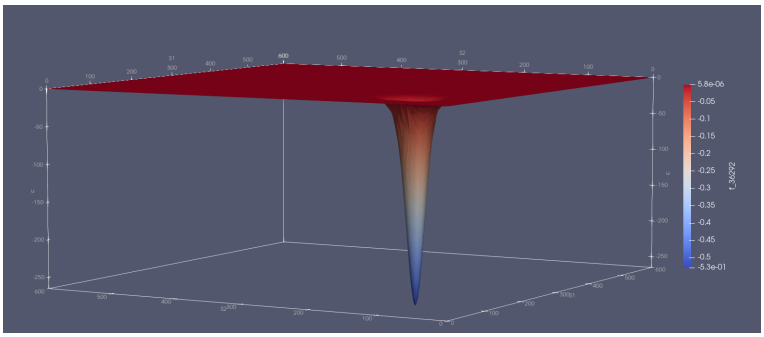

Figure 6: The solution of the dual problem.

The initial mesh is 2×2 and Tables 1 and 2 will present the error (e) and the cumulative time (T_C) for each iteration (i). Table 1 shows the actual mean error while table 2 shows the estimated mean error for each iteration. Notice that, when the uniform, residual, and local mesh refinements are used, there is no need for calculating the dual problem. The times presented are after excluding the unnecessary computations. Also, the time step scales with the square of the size of the smallest element in the mesh. Because the time steps get fairly small, we use LU-factorization to solve the problem. This works because the left-hand side matrix does not change between time steps and as such we can save the factorization and thus calculate every time step after the first one faster than with an iterative solver.

Two quantities are measured to compare the methods. Firstly, the actual mean error in nine points around the interesting point and the time it takes to get there for each iteration. The second is the estimated error and the time it takes to get there. The estimated error is the average of the absolute difference between the values at this iteration and the old iteration. The "real value" that we compare with is a very accurate solution given by Benchop.

The parameters are chosen as $r = 0.03$, $\sigma_1 = \sigma_2 = 0.15$, $\rho = 0.5$ and $K = 0$.

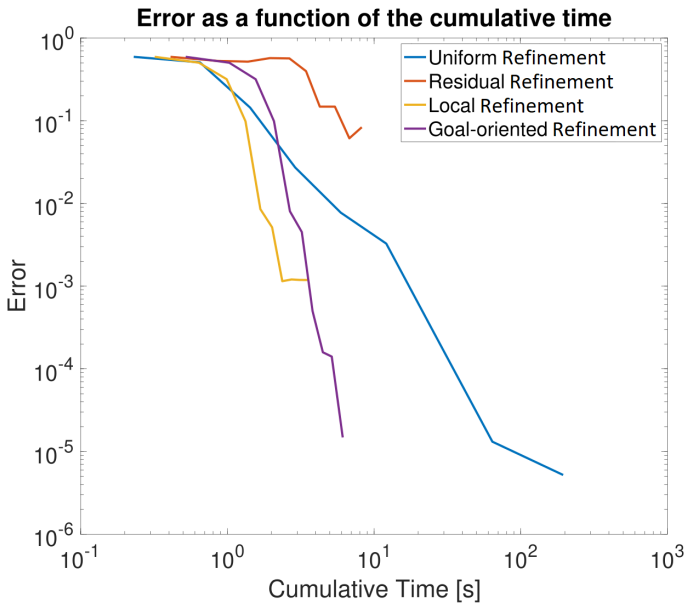

Figure 7: The actual mean error as a function of run time for the 2D problem.

	Uniform		Residual		Local		Goal-oriented	
i	e	$T_C[s]$	e	$T_C[s]$	e	$T_C[s]$	e	$T_C[s]$
0	5.93e-01	0.036	5.93e-01	0.053	5.92e-01	0.050	5.92e-01	0.075
1	5.08e-01	0.092	5.29e-01	0.119	5.03e-01	0.105	5.03e-01	0.145
2	1.44e-01	0.188	5.18e-01	0.190	3.17e-01	0.163	3.17e-01	0.216
3	2.69e-02	0.373	5.72e-01	0.265	9.82e-02	0.216	9.84e-02	0.286
4	7.91e-03	0.792	5.67e-01	0.357	8.60e-03	0.278	8.12e-03	0.374
5	3.34e-03	2.21	3.98e-01	0.463	5.21e-03	0.373	4.58e-03	0.510
6	2.61e-04	9.82	1.48e-01	0.593	1.51e-03	0.586	5.47e-04	0.836
7	1.85e-05	70.4	1.48e-02	0.758	1.21e-03	1.27	1.80e-04	1.46
8	1.32e-06	687	6.20e-02	1.33	1.20e-03	3.89	1.42e-04	2.68
9	-	-	8.42e-03	1.60	1.19e-03	14.7	2.17e-05	5.32

Table 1: The actual mean error and cumulative time for each iteration for the four refinement methods.

The goal-oriented mesh refinement reaches the desired error of less than 1e-4 around 10 times faster than the uniform refinement, which can be seen in Figure 7. The other two refinements do not reach that error tolerance at all. The meshes can be visualized to show

	Uniform		Residual		Local		Goal-oriented	
i	$\hat{e}$	$T_C[s]$	$\hat{e}$	$T_C[s]$	$\hat{e}$	$T_C[s]$	$\hat{e}$	$T_C[s]$
0	-	0.036	-	0.053	-	0.050	-	0.075
1	1.76e-01	0.092	1.38e-01	0.119	1.84e-01	0.105	1.84e-01	0.145
2	4.27e-01	0.188	2.34e-02	0.190	2.75e-01	0.163	2.75e-01	0.216
3	1.21e-01	0.373	1.30e-01	0.265	2.44e-01	0.216	2.44e-01	0.286
4	3.40e-02	0.792	1.16e-02	0.357	1.05e-01	0.278	1.03e-01	0.374
5	5.61e-03	2.21	2.86e-01	0.463	5.70e-03	0.373	5.78e-03	0.510
6	3.08e-03	9.82	2.94e-01	0.593	3.70e-03	0.586	4.03e-03	0.836
7	2.43e-04	70.4	3.70e-05	0.758	3.00e-04	1.27	3.94e-04	1.46
8	1.74e-05	687	9.19e-02	1.33	2.03e-05	3.89	5.95e-05	2.68
9	-	-	2.43e-02	1.60	1.56e-06	14.7	1.25e-04	5.32

Table 2: The estimated mean error and cumulative time for each iteration for the four refinement methods.

what the different refinements have been prioritized to refine. This is done in Figures 8 - 10. The local method and the goal-oriented refinement method refine similarly, but the local method will always refine close to our reference value while the goal-oriented method will refine further away when it seems better. The residual-based method refines

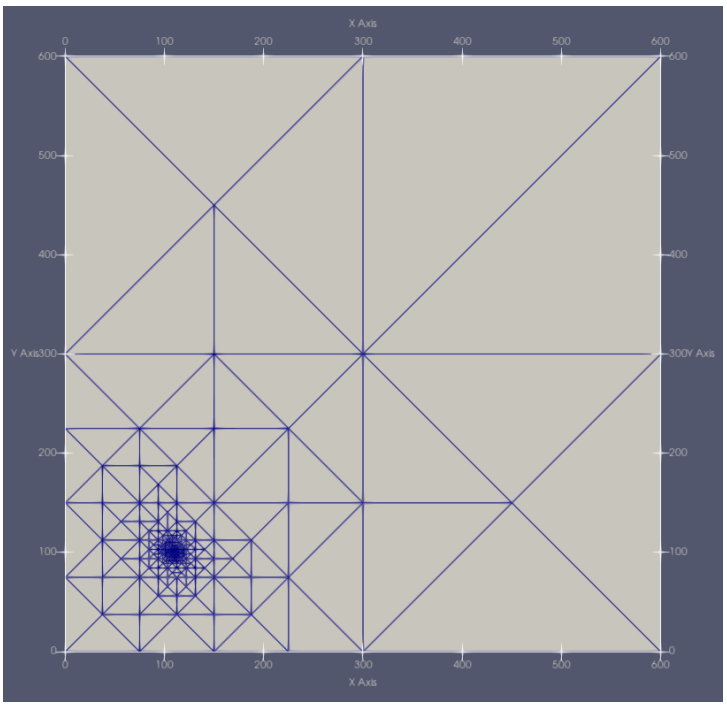

Figure 8: The final mesh after refinement when using the local refinement method.

a bit over the entire domain, though focusing mostly on the northeast corner, where the residual is the largest. The residual is in general largest along the diagonal since the initial condition is discontinuous in the first derivative there and it gets larger in the north-west corner due to the spatially dependent coefficients. It is possible to avoid this by log-transforming the Black-Scholes equation, as in (9). This gives constant coefficients but it is unlikely that this would have any meaningful effect on the usefulness of the

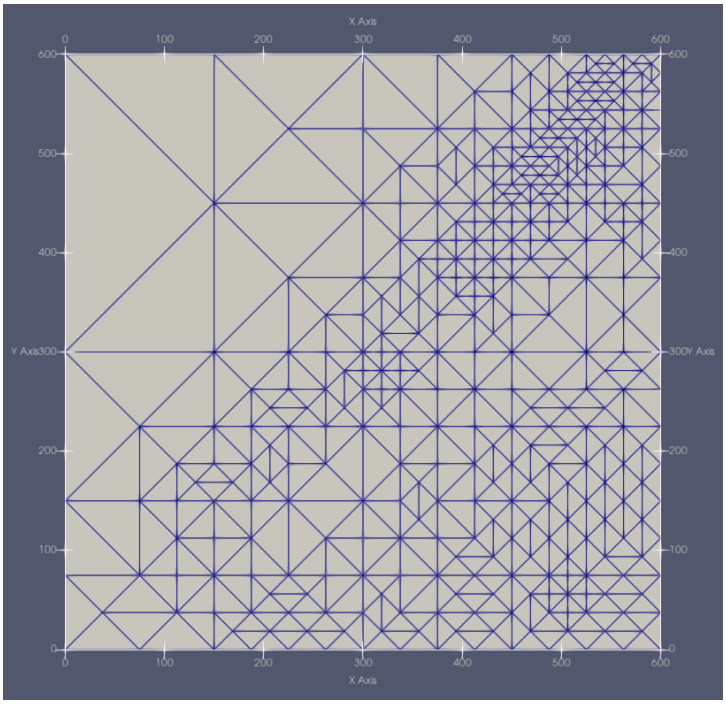

Figure 9: The final mesh after refinement when using the residual-based refinement method.

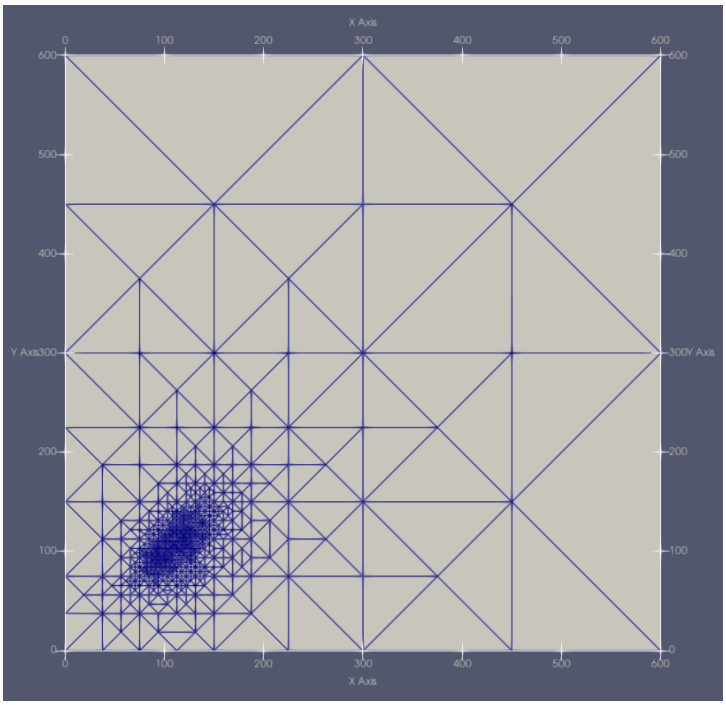

Figure 10: The final mesh after refinement when using the goal-oriented refinement method.

residual-based method. It is worth noting that the residual-based method works quite well when we want to minimize the error over the entire domain. However, since we do not care much about accuracy in the vast majority of the domain, it becomes somewhat

inefficient in our case.

Compared to the local and residual, the goal-oriented mesh appears to be a combination of the residual-based method and the local method. It has the properties of local refinement since it mostly refines near the area of interest, but it also does this in an ellipse with a major axis along the strike. This orientation of the ellipse probably comes from the residual term in (36) since the residual-based method is also refined along the strike. This makes sense since the discontinuity in the derivative is at strike for time 0, it is here that most of the residual will be when it is not close to a boundary.

Now, why does the goal-oriented refinement focus on refining close to the point of interest? Surely, it makes sense to refine there, but there is no "local" term in (36). However, the gradient term of the dual solution will in this case act as the local term. The dual solution will look like a pillar that rises around the interesting point. Therefore, the gradient around this area will be high and this is where the local part comes into play. This is because of the target function that we have chosen.

Even if equation (36) includes an integral over time, in our case it will not change the outcome of the refinement significantly. Since the drift term is quite small, the "solution" will not travel around in the domain. If it would, the goal-oriented refinement could have been refined at the track where the solution would travel such that the error does not propagate when the solution moves. However, it is not only the drift term that can trigger the benefits of a time integral. Since we have a diffusion term, it might be beneficial to have it refined where the solution will diffuse a lot at early times to not make the error propagate through the diffusion.

We see this effect if another payoff is used, something we call the Gaussian derivative, which is defined by (42). If we use high volatility of 0.3, no correlation, and no interest rate, then a Gaussian which will diffuse over time will be obtained, as seen in figure 11. Figure 12 shows that it prioritizes where the solution is great early such that the error does not snowball from a bad early estimation of the diffusion.

$$\exp(-c((s_1 - s_{1,0})^2 + (s_2 - s_{2,0})^2)). \tag{42}$$

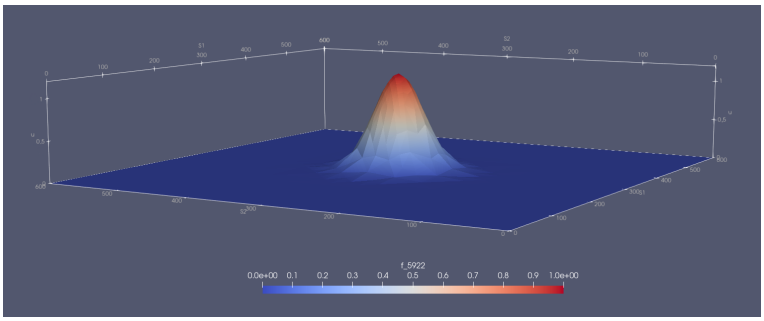

Figure 11: The initial condition of the Gaussian derivative.

Although the uniform refinement also reaches the target accuracy, it does this a lot slower. Remember that computational speed is also important when pricing options since the

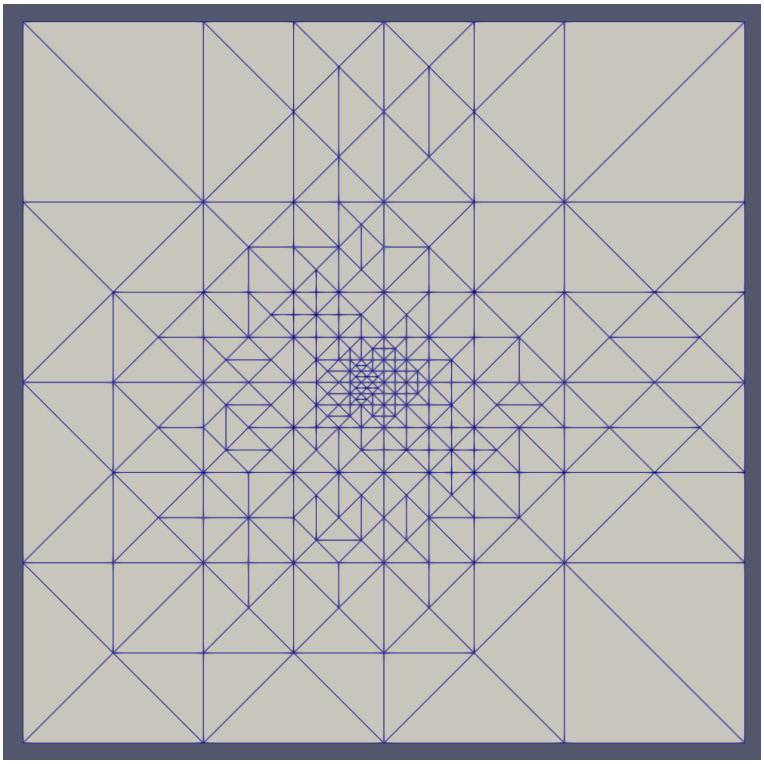

Figure 12: The mesh refinement of the Gaussian derivative.

market shifts so quickly. An accurate pricing a minute too late might no longer be accurate. Therefore, the goal-oriented method is to be recommended.

5.1.2 Accuracy and correlation of assets

In this subsection, we look at how these mesh refinements differ in efficiency when the correlation of the assets changes. The parameters are chosen as $r = 0.03$, $\sigma_1 = \sigma_2 = 0.15$, $K = 0$, and ρ will vary. We measure the time it takes to reach 1e-4 for the actual mean error, and how long it takes for it to reach the break condition.

We can see that the goal-oriented mesh refinement is the method that solves the task in a reasonable time for most correlations, see Figure 13. The time to reach the estimated accuracy can be seen in Figure 14.

When the correlation is high, the solution will be closer to the payout than if the correlation is low. If the correlation is exactly 1, then the solution will be the exact same solution as the payout. This is because a correlation of 1 implies that any change in s_1 is followed by an identical change in s_2. As such, their difference will always be the same and the option value is constant in time. Therefore, the higher the correlation, the less of the discontinuity in the derivative will diffuse away, and it is harder to reach our target accuracy.

Another aspect to monitor is whether the time for the error estimate to reach below the threshold is greater or less than the target time. If the estimation time is less, then the algorithm will stop before the actual accuracy is met, which is not good. The goal-

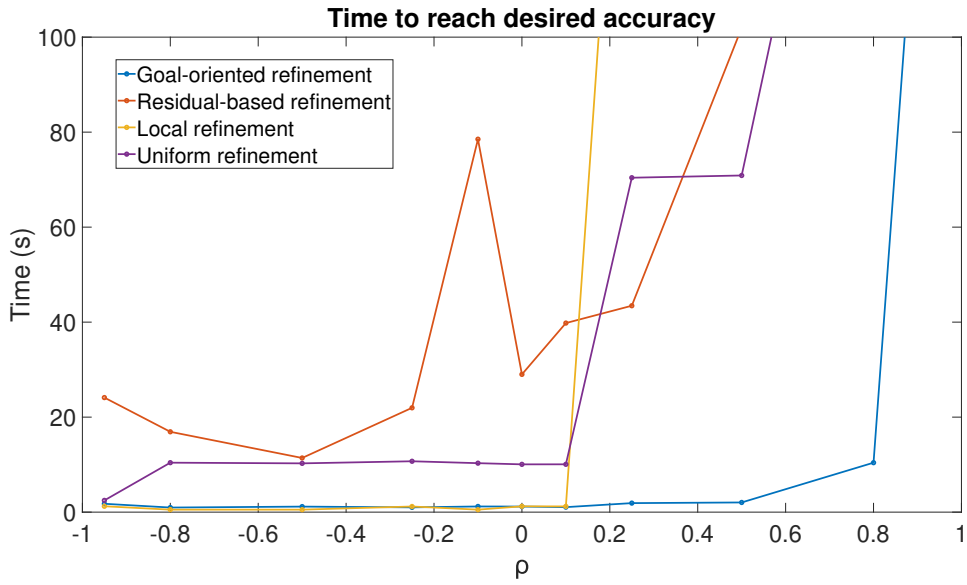

Figure 13: Time to reach target accuracy for different correlations.

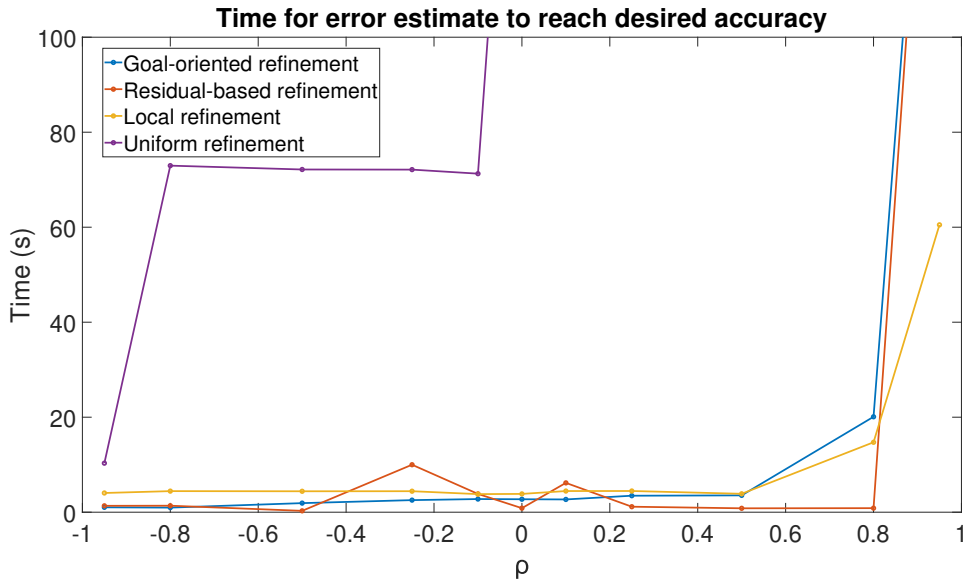

Figure 14: Time to reach estimated accuracy for different correlations.

oriented method will, for almost all correlations, reach the desired accuracy when using the break condition, while residual refinement will not. It will assume that its accuracy is better than it actually is and thus break too early which explains the short runtimes in Figure 14. Similar results can be seen for the local refinement, the break condition is fine for negative correlations, but not fine when the correlation becomes larger than 0.1. These relations are shown in Table 3, where T_a is the cumulative time to reach the actual accuracy of 1e-4 and T_e is the cumulative time to reach the error estimate of 5e-5.

To be able to use this break condition, the only adaptive mesh refinement that works is goal-oriented.

	Uniform		Residual		Local		Goal-oriented	
ρ	$T_a[s]$	$T_e[s]$	$T_a[s]$	$T_e[s]$	$T_a[s]$	$T_e[s]$	$T_a[s]$	$T_e[s]$
-0.95	2.47	10.3	24.1	1.34	1.24	4.04	1.76	1.03
-0.8	10.4	73.0	16.9	1.37	0.522	4.04	0.966	0.966
-0.5	10.3	72.2	11.4	0.29	0.522	4.44	1.16	1.91
-0.25	10.7	72.1	22.0	10.0	1.19	4.40	1.00	2.54
-0.1	10.3	71.3	78.5	3.85	0.521	4.42	1.19	2.77
0	10.1	200+	29.0	0.85	1.22	3.81	1.17	2.73
0.1	10.1	200+	39.8	6.17	1.22	3.85	1.05	2.69
0.25	70.4	200+	43.5	1.16	200+	4.45	1.90	3.47
0.5	70.9	200+	102	0.82	200+	4.46	2.04	3.54
0.8	200+	200+	200+	0.85	200+	14.7	10.4	20.1
0.95	200+	200+	200+	200+	200+	60.5	200+	200+

Table 3: Time to reach both the actual accuracy of 1e-4 and time to reach the error estimate of 5e-5.

5.2 LU-factorization or Iterative solvers

In this section, we shortly mention the difference between choosing an LU solver, or an iterative solver when solving the PDE. An LU solver solves the first time step slowly, but all the other time steps fast. An iterative solver solves all time steps equally fast and is faster than the first time step of LU, but slower than all the rest of the time steps of LU. If the system becomes too large, then the LU decomposition is prohibitively expensive, and thus iterative solvers are necessary.

When the system becomes large, iterative solvers become appropriate, and when the time step becomes small, LU becomes more appropriate. However, in our case, when the system becomes larger, the time step usually has to be smaller to get an accurate solution. The time step has to scale with the smallest element in our domain, and depending on how the goal-oriented mesh refinement chooses to refine, the time step can quickly become small. Therefore, a priori, it is not clear whether LU or an iterative solver should be chosen.

In the end, we have chosen to use an LU solver for the 2D problem, and an iterative solver for the 3D problem. The iterative method used in this project is the BiCGSTAB-method. Note that there are many iterative solvers and many options, such as preconditioners, that can benefit the iterative method. Thus, in general, it is hard to say that LU is more efficient for the 2D case.

5.3 3D Results

For the 3D problem our point of interest will be at $(s_1, s_2, s_3) = (300, 300, 300)$. The results for 3D were similar but it takes a longer time to compute. Here it is not possible to use uniform refinement since the extra dimension makes the problem scale in size extremely fast and it very quickly becomes so large that it takes too long to compute the solution and too much memory to store. Thus, it is only viable to use methods that refine only parts of the domain. As is the case for the 2D problem, the residual-based and

local refinement do not need to calculate the dual problem and the local does not need to calculate the residual. This saves time. The local refinement is however still slower at the same number of iterations because it prefers refining the same subdomain all the time and as such ends up with much smaller elements, which makes it require much smaller time steps. While the local refinement does reach the desired accuracy of 1e-4 it does so significantly slower than our adaptive method, which can be seen in Figure 15 or Table 4. The residual-based method, while still insufficient, performs a lot better. The reason for this may be that the domain where the residual is large is closer to and overlaps more with the point of interest than it does for the 2D spread option. Refinement-wise, the results look very similar with the residual spreading out its refinement, trying to minimize the residual over the entire domain, the local-based focuses entirely on refining the point of interest. Finally, the goal-oriented refinement focuses mostly around the point of interest but also refines a bit further away if it judges that it is beneficial for the error in the point of interest. A cross-section of the mesh refinements can be seen in Figures 17 - 19.

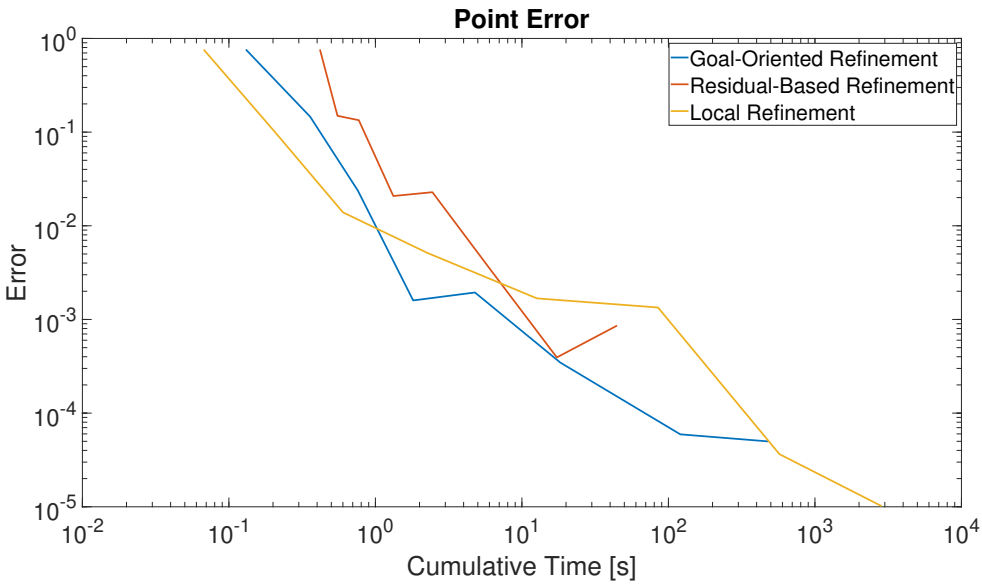

Figure 15: The point error as a function of time for the 3D problem.

	Residual		Local		Goal-oriented	
i	e_{point}	$T_C[s]$	e_{point}	$T_C[s]$	e_{point}	$T_C[s]$
1	7.620e-01	0.418	7.620e-01	0.067	7.620e-01	0.131
2	1.493e-01	0.551	9.987e-02	0.207	1.462e-01	0.358
3	1.340e-01	0.770	1.391e-02	0.600	2.370e-02	0.760
4	2.078e-02	1.33	5.134e-03	2.26	1.596e-03	1.81
5	2.283e-02	2.45	1.678e-03	12.7	1.940e-03	4.78
6	3.680e-03	5.87	1.342e-03	84.9	3.468e-04	18.3
7	3.932e-04	17.3	3.647e-05	572	5.937e-05	120
8	8.600e-04	44.6	1.016e-05	2850	4.989e-05	479

Table 4: Point error vs time for the 3D problem.

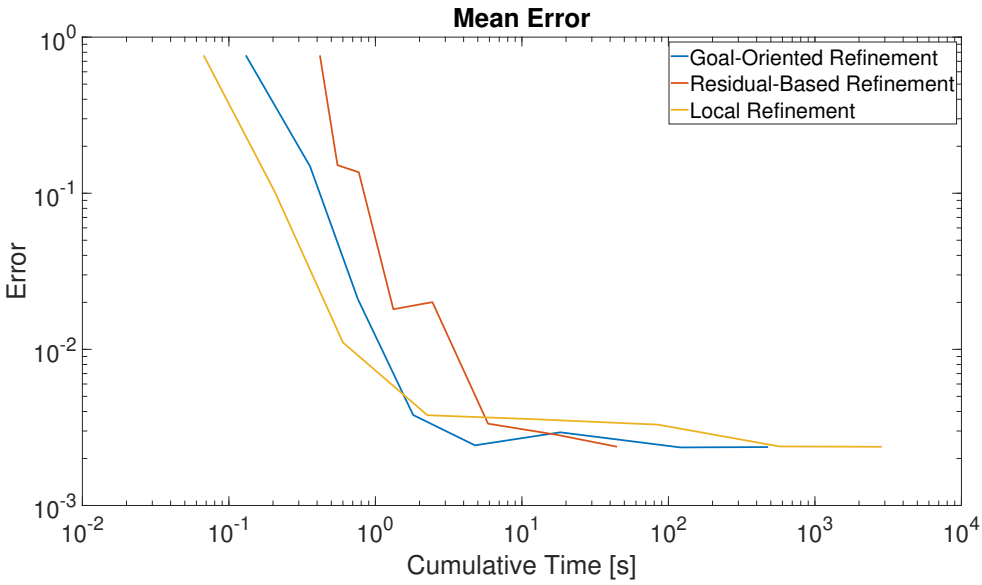

Figure 16: The mean error as a function of time for the 3D problem.

	Residual		Local		Goal-oriented	
i	e_{mean}	$T_C[s]$	e_{mean}	$T_C[s]$	e_{mean}	$T_C[s]$
1	7.639e-01	0.418	7.639e-01	0.067	7.639e-01	0.131
2	1.151e-01	0.551	1.013e-01	0.207	1.482e-01	0.358
3	1.362e-01	0.770	1.106e-02	0.600	2.096e-02	0.760
4	1.807e-02	1.33	3.787e-03	2.26	3.798e-03	1.81
5	2.007e-02	2.45	3.563e-03	12.7	2.428e-03	4.78
6	3.334e-03	5.87	3.295e-03	84.9	2.942e-03	18.3
7	2.836e-03	17.3	2.388e-03	572	2.355e-03	120
8	2.378e-03	44.6	2.377e-03	2850	2.367e-03	479

Table 5: Mean error vs time for the 3D problem.

5.4 Non-linear results

For the non-linear case, the linear case can be obtained by setting the constant a to zero. Therefore, a comparison could be made between how the resulting solution differs from the linear case by looking at different values of a.

However, the method of frozen coefficients does not converge except when the time step is too small to be practical and therefore the conclusion is that FEM can not handle this type of non-linear case using this method. Another approach has to be considered.

One approach worth trying could be to use fixed-point iterations instead of locking one of the second derivatives. Instead of locking it to the previous time step, fixed-point iterations could be used and in that way update the "locked" second derivative for every iteration, while still being in the same time step. This has not been investigated in the current study.

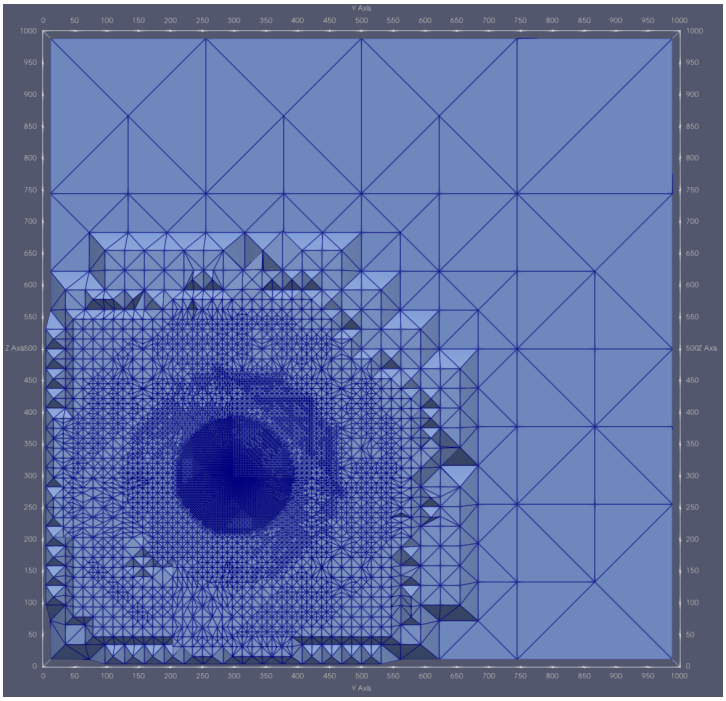

Figure 17: A cross section of the final mesh after refinement when using the local refinement method.

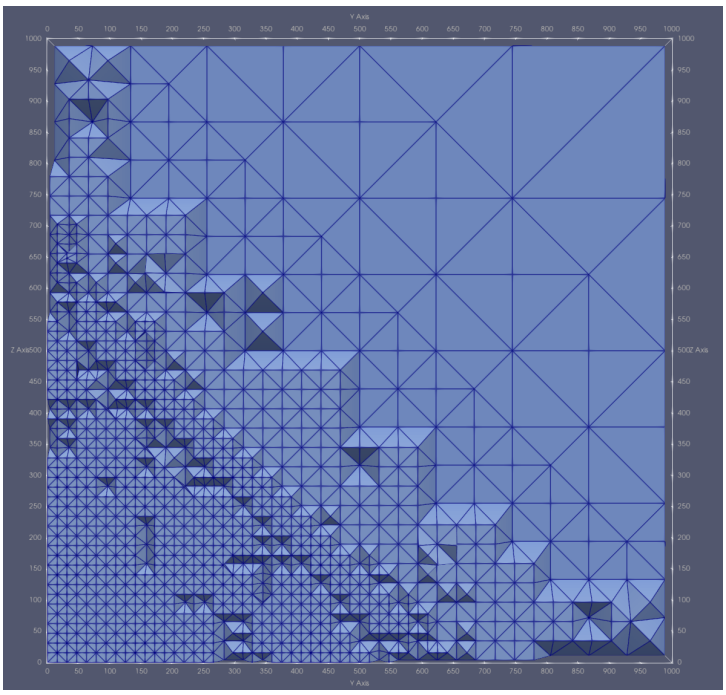

Figure 18: A cross section of the final mesh after refinement when using the residual based refinement method.

6 Conclusion

In conclusion, we can say that the project is successful. The goal-oriented method performed well compared to the other mesh-refinement methods, both in that it could reach

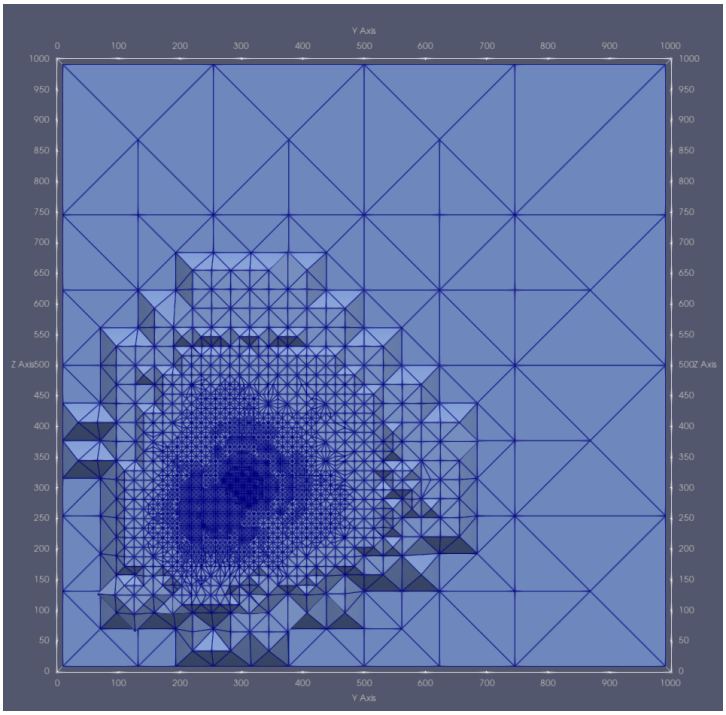

Figure 19: A cross section of the final mesh after refinement when using the goal-oriented refinement method.

the target accuracy, which local and residual refinement could not, and that it outperforms the uniform refinement in computational speed.

For 3D the results are similar. The goal-oriented mesh refinement outperforms the other methods in speed and accuracy. For the non-linear case, another approach must be used to solve the task in a reasonable time. This is discussed more in the next section.

7 What to do next?

The results of the project open new questions to be investigated further. In this section, three different continuations that can be done to this project are listed here.

7.1 Non-linear problems

The first direction that this project could be extended to is to look more at the non-linear problem. One way of fixing the non-converging issue would be not to lock one of the second derivatives to the earlier time-step but instead use fixed point iterations. For every iteration in the iterative method, one of the second derivatives could be locked as the second derivative of the previous iteration and not the previous time step. This would make the solutions more similar and the method might have an easier time to converge.

7.2 More options in 2D / 3D

Another extension of this project would be to consider more options in 2D or 3D and see if goal-oriented mesh refinement is also successful for these options. Path-dependent

options such as the Asian options or the non-linear American option could be some of the alternatives.

7.3 Time-dependent / Stochastic volatility

In the real world, it is not always the case that the volatility is constant between the current time and the time of maturity. Therefore, a third extension could be to use time-dependent or stochastic volatility. When using stochastic volatility, another dimension has to be added for each asset which makes it even more important to have a good adaptive method. Finding appropriate boundary conditions will also become harder. It would be insightful to see whether the goal-oriented mesh refinement could handle this task as well.

References

[1] Yves Achdou and Olivier Pironneau. "Finite element methods for option pricing". In: *Université Pierre et Marie Curie* (2007), pp. 1–12.

[2] Martin S. Alnæs et al. "The FEniCS Project Version 1.5". In: *Archive of Numerical Software 3* 3.100 (2015). DOI: doi.org/10.11588/ans.2015.100.20553.

[3] Tomas Björk. *Arbitrage theory in continuous time*. Oxford university press, 2009.

[4] Fischer Black and Myron Scholes. "The pricing of options and corporate liabilities". In: *World Scientific Reference on Contingent Claims Analysis in Corporate Finance: Volume 1: Foundations of CCA and Equity Valuation*. World Scientific, 2019, pp. 3–21.

[5] Aicha Driouch and Hassan Al Moatassime. "Multigrid Method for a Two Dimensional Fully Nonlinear Black-Scholes Equation with a Nonlinear Volatility Function". In: *J. Math. Study* 53.3 (2020), pp. 247–264.

[6] E. Kirk. "Correlation in the energy markets". In: *Managing Energy Price Risk, V. Kaminski* 1 (1995), pp. 71–78.

[7] Karl Ljungkvist. "Techniques for finite element methods on modern processors". PhD thesis. Uppsala University, 2015.

[8] Rüdiger Seydel and Rudiger Seydel. *Tools for computational finance*. Vol. 3. Springer, 2006.

[9] Lina von Sydow et al. "The BENCHmarking project in Option Pricing". In: *International Journal of Computer Mathematics* 93 (2015), pp. 2361–2379. DOI: https://doi.org/10.1080/00207160.2015.1072172.